Shift left Security

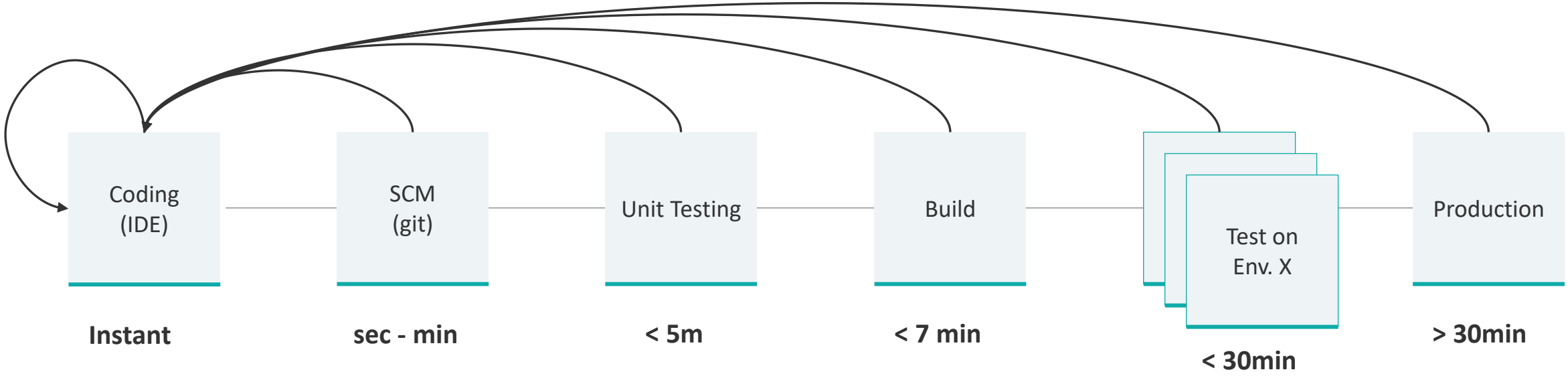
17.06.2022, Florian Kammermann

swisscom

Agenda

1. The Fast Feedback Loop
2. Security in the Software Development Life Cycle (SDLC)
3. Trust in the Software Development Life Cycle (SDLC)
4. Securing Code and Configuration, Artifacts
5. Prevent of Attacks in the Software Development Life Cycle (SDLC)

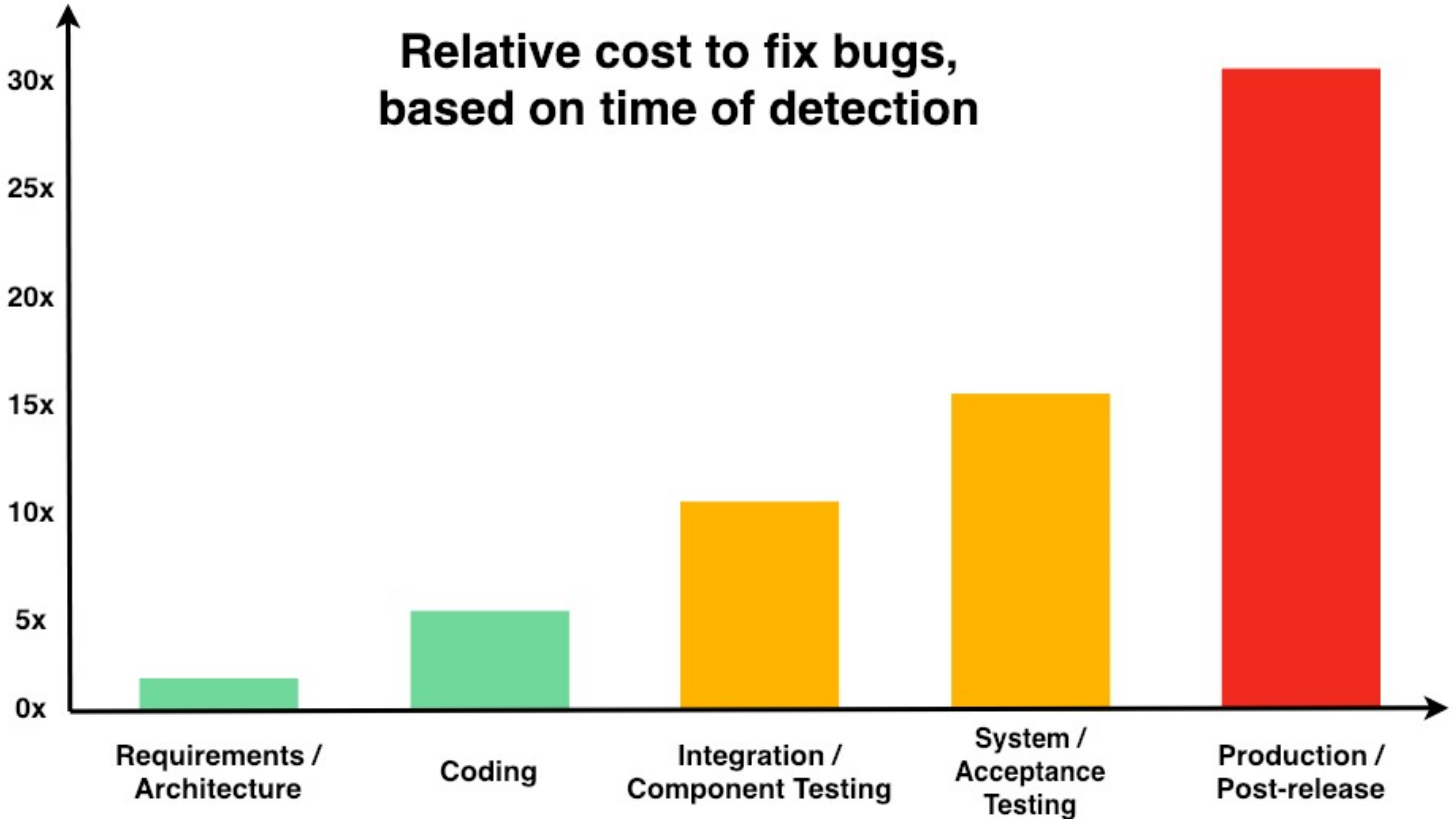
Fast Feedback Loop



The **more left** we hold the **sensor**,
the **faster** we get the **feedback**,
the **faster** we **fix** it,
the **lower** the **cost**,
the **lower** the **security exposure**

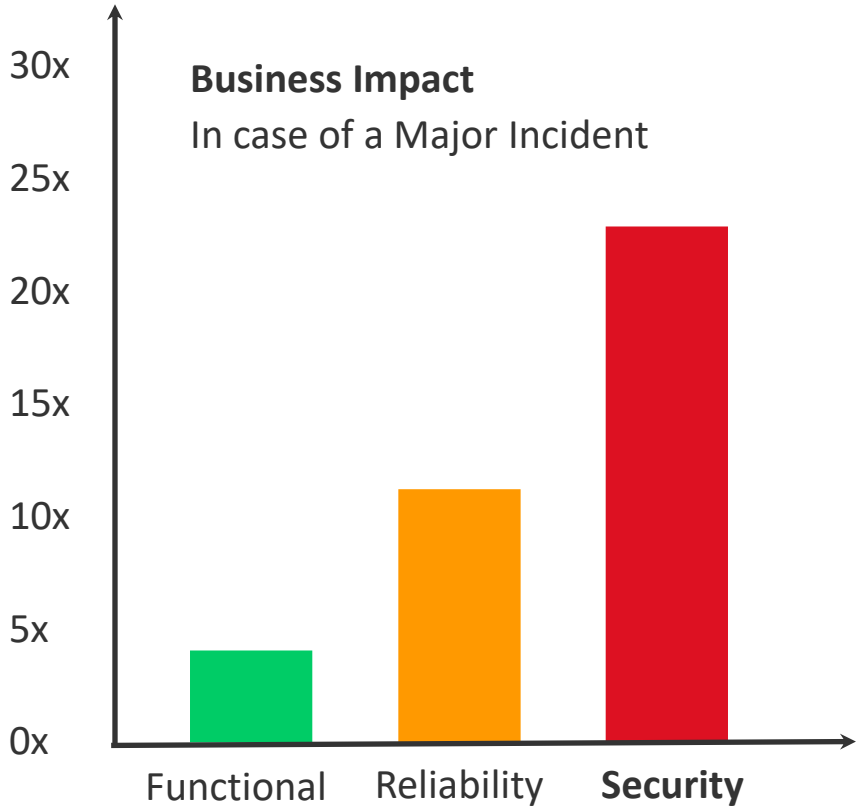
Fast Feedback Loop – The Business View

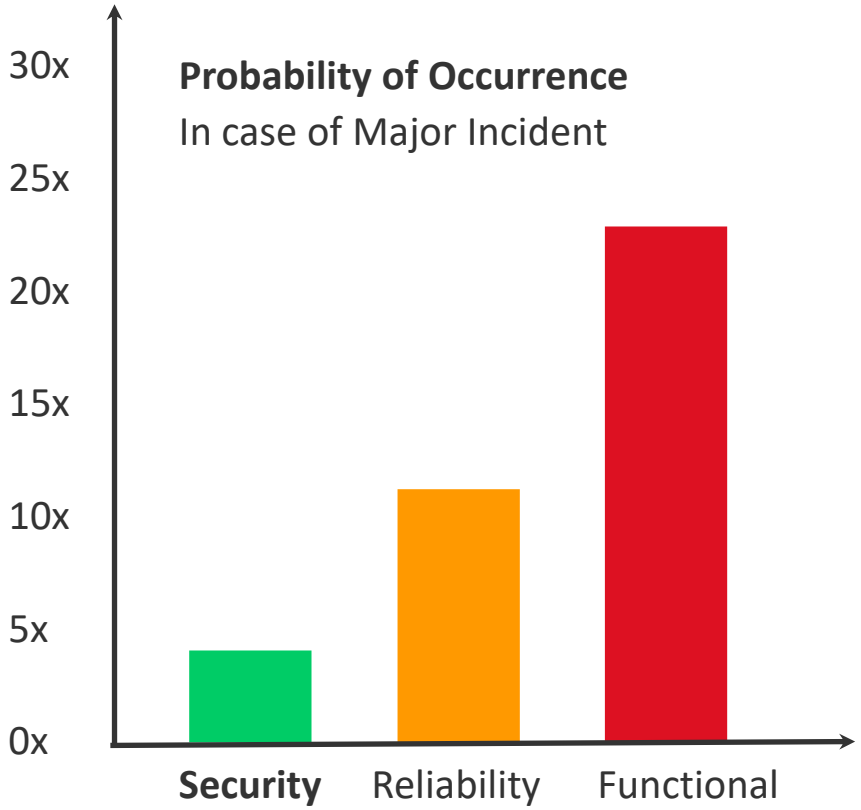
Cost per defect (functional, reliability, security)



Source: **NIST**

Incident Types Impact / Probability



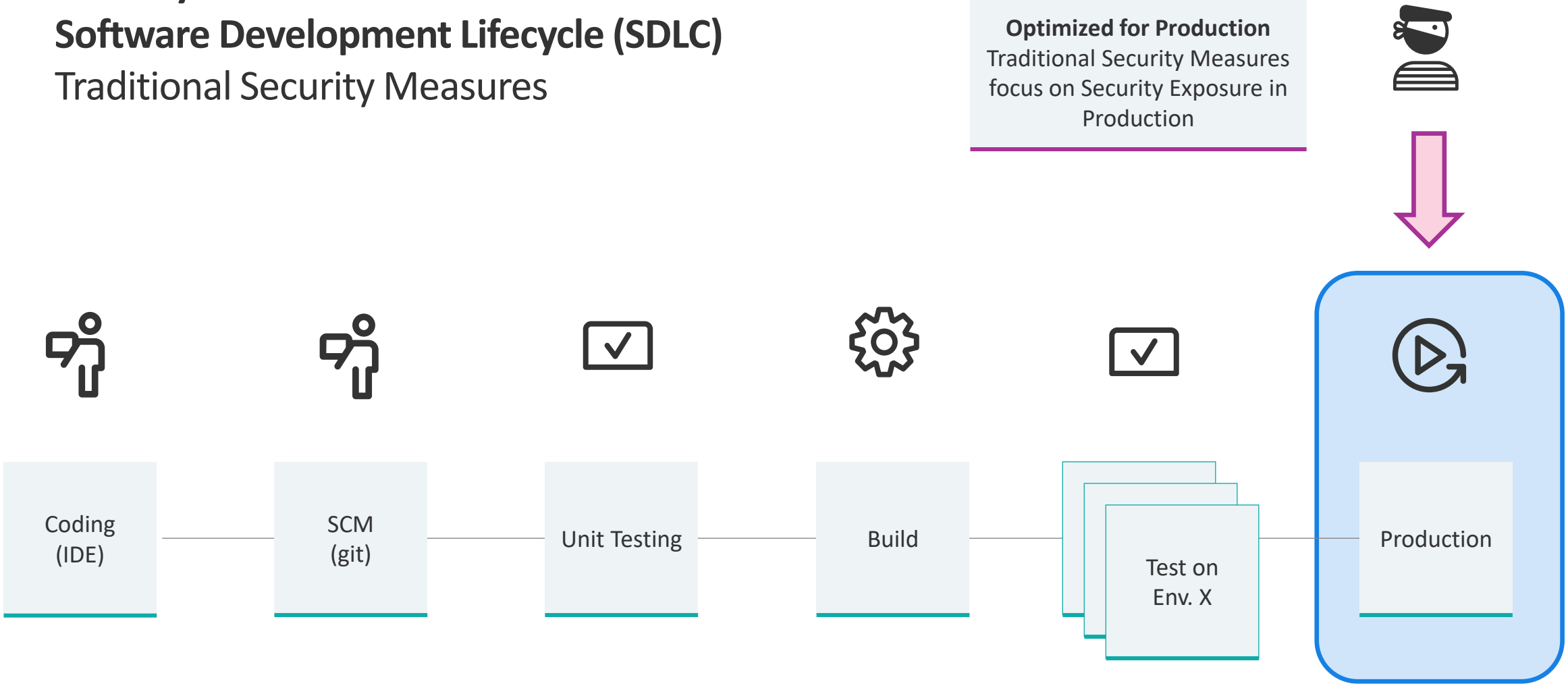


Optimizing for Security is a difficult Business Case,
we have **low probability** with **high impact**.

Security in the Software Development Lifecycle (SDLC)

Traditional Security Measures

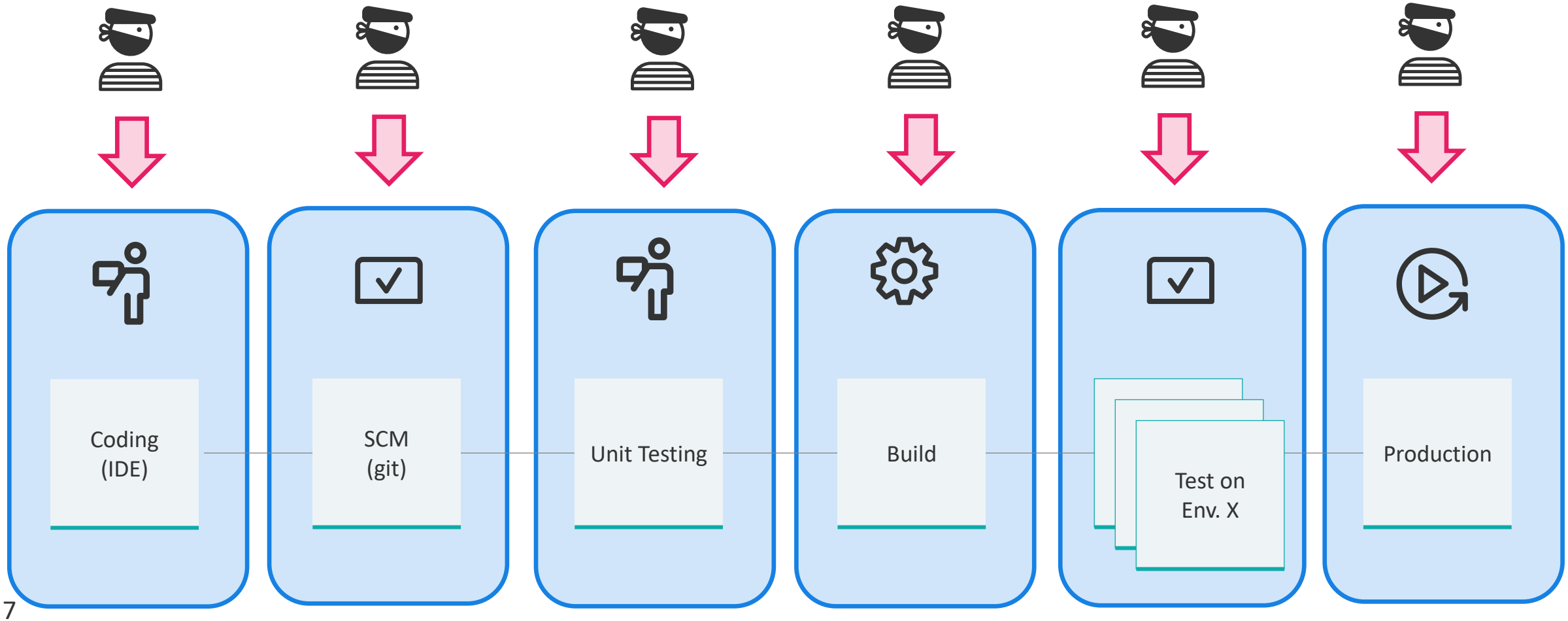
Optimized for Production
Traditional Security Measures focus on Security Exposure in Production



Security in the Software Development Lifecycle (SDLC)

SSDLC Security Measures

Focus on the whole SDLC
Digitalized Production assumes that every step has its security posture



The New Reality – Attack Vectors on the Software Development Lifecycle

OWASP Top 10

- 04 Insecure Design
- 05 Security Misconfiguration
- 06 Vulnerable and outdated Components
- 08 Software and data integrity failures

NCSC (Nationales Zentrum für Cybersecurity)

- Angesichts der zunehmenden Komplexität von Hard- und Software sowie der fortschreitenden Digitalisierung der Gesellschaft stellen nicht zuletzt **Software-Abhängigkeiten** eine **grosse Herausforderung für die Sicherheit** von Unternehmen dar.
- In den letzten Jahren hat die «US National Telecommunications and Information Administration (NTIA)» mit Partnern an der Entwicklung einer **«Software Bill of Materials (SBOM)»** gearbeitet.

Executive Order improving the Nations Cybersecurity

Shift left Security
CISA Model part of **Zero Trust Application Pillar**
(CISA = Cybersecurity & Infrastructure Security Agency)

Security in the Software Development Lifecycle

How Security is applied to the SDLC

Digitalized Production Security Measures (digital Identities and Bill of Materials)

Traditional Security Measures (SAST, DAST / PEN Test)

Trust

How we create Trust into the Software Development Lifecycle

implicit

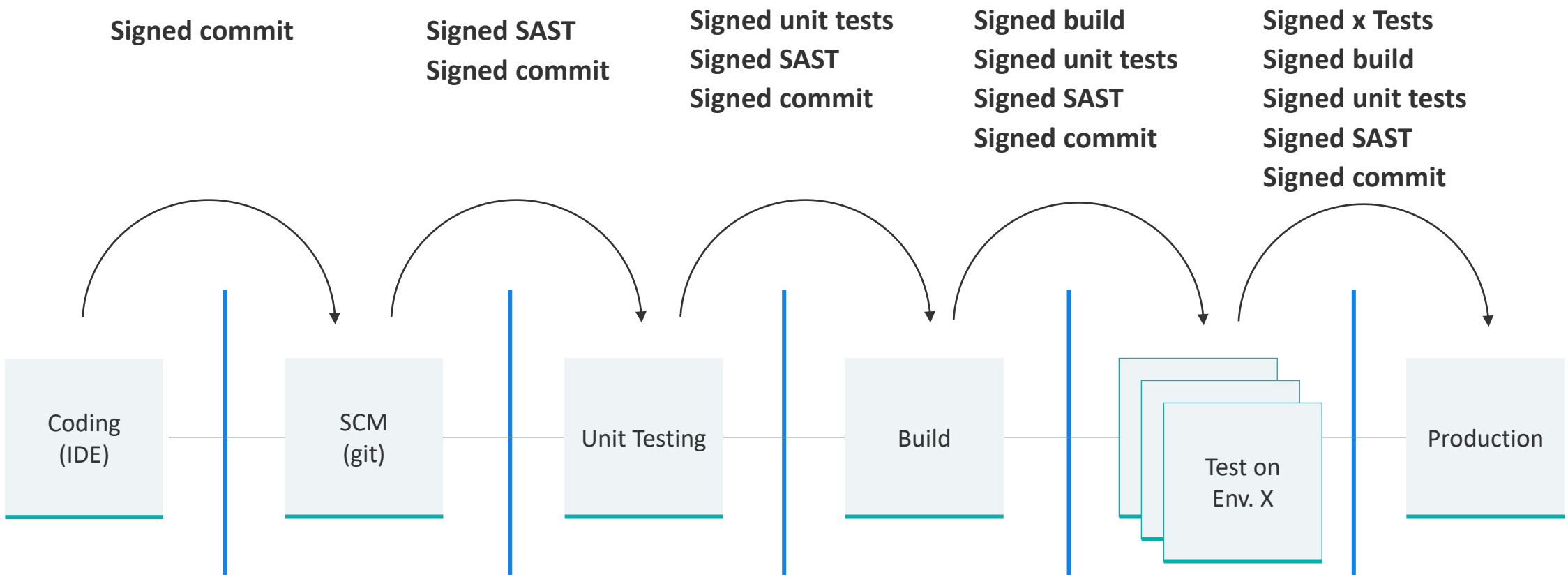
Implicit Trust is created by ticketing systems, by verbal communication by email or chat. Implicit trust is not auditable and can not be traced back programmatically. Implicit trust is not verifiable. Implicit trust doesn’t protect from tampering.

explicit

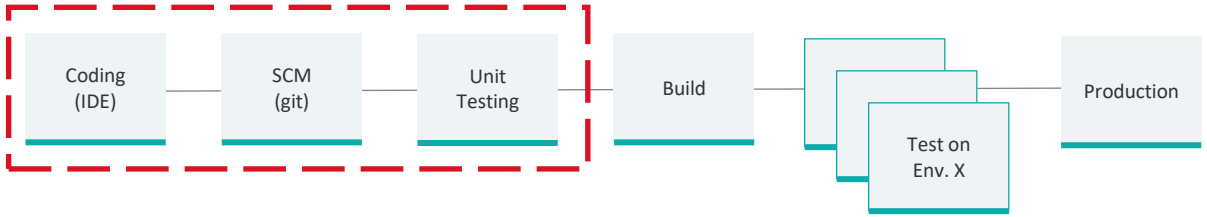
Explicit Trust is auditable and verifiable by digital evidence. Explicit trust also comes with digitally represented parties, which took an action in the SSDLC (Secure Software Development Lifecycle). Explicit Trust creates a chain of trust, which can be verified by external parties.

Accumulate explicit Trust over the Pipeline Stages

digitally signed propagation

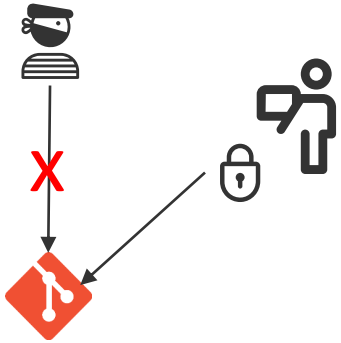


Securing Code and Configuration



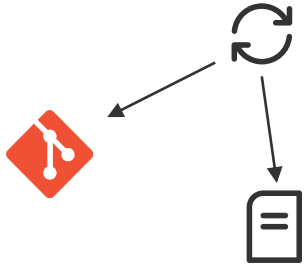
Commit Authenticity

A foundation of securing source code is the ability to authenticate contributor-added commits to the repository. Unsigned or unknown signatures have to be rejected.



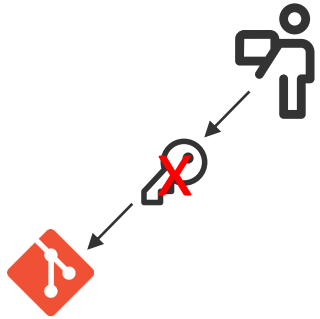
Identify malicious code early

Use SAST to scan the code for vulnerabilities. SAST can find code segments which can lead to possible security exposure.
https://cwe.mitre.org/



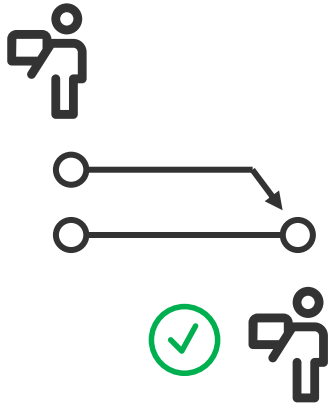
Avoid exposing sensitive information

It's still a problem, that secrets can make it into the code base. Use pre-commit hooks to avoid sensitive information in the central git repository.

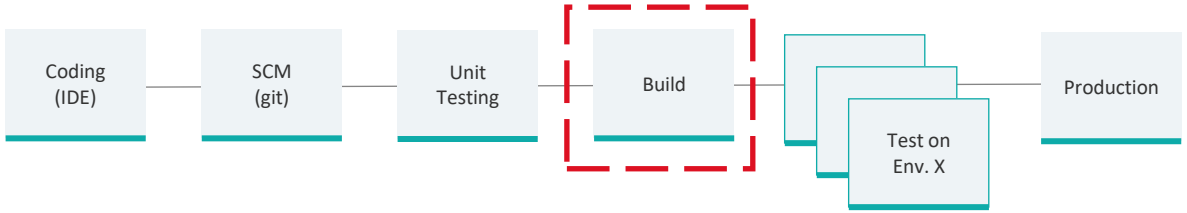


Flawless Change Management

Review Code Changes by qualified Coworkers. Create short lived branches to bring new code to main. Avoid long lived feature branches.



Securing Artifacts before Deployment

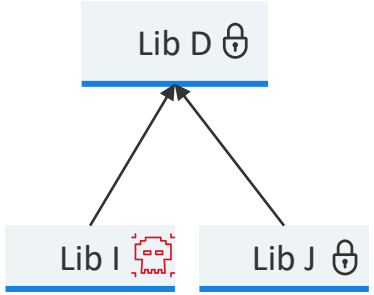


Artifact Repositories

Artifact Repositories play a central role in the SDLC. The accumulated Trust and Immutability needs to be guaranteed by Artifact Repositories. They are also the gate for incoming artifacts from central repositories.

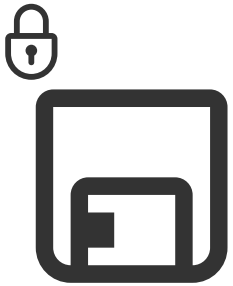
Identify malicious dependencies early

Analyses artifacts to detect known software components and identify any associated vulnerabilities. Scanning complements SAST by finding vulnerabilities not detectable by scanning source code and can also help to build the SBOM.



Managed base Images (VM/OCI)

VM images should be treated the same as any artifact. They should go through all the stages of the SSDLC. For OCI Images, distroless images are to favor, to remove all the non-essential operating system dependencies.

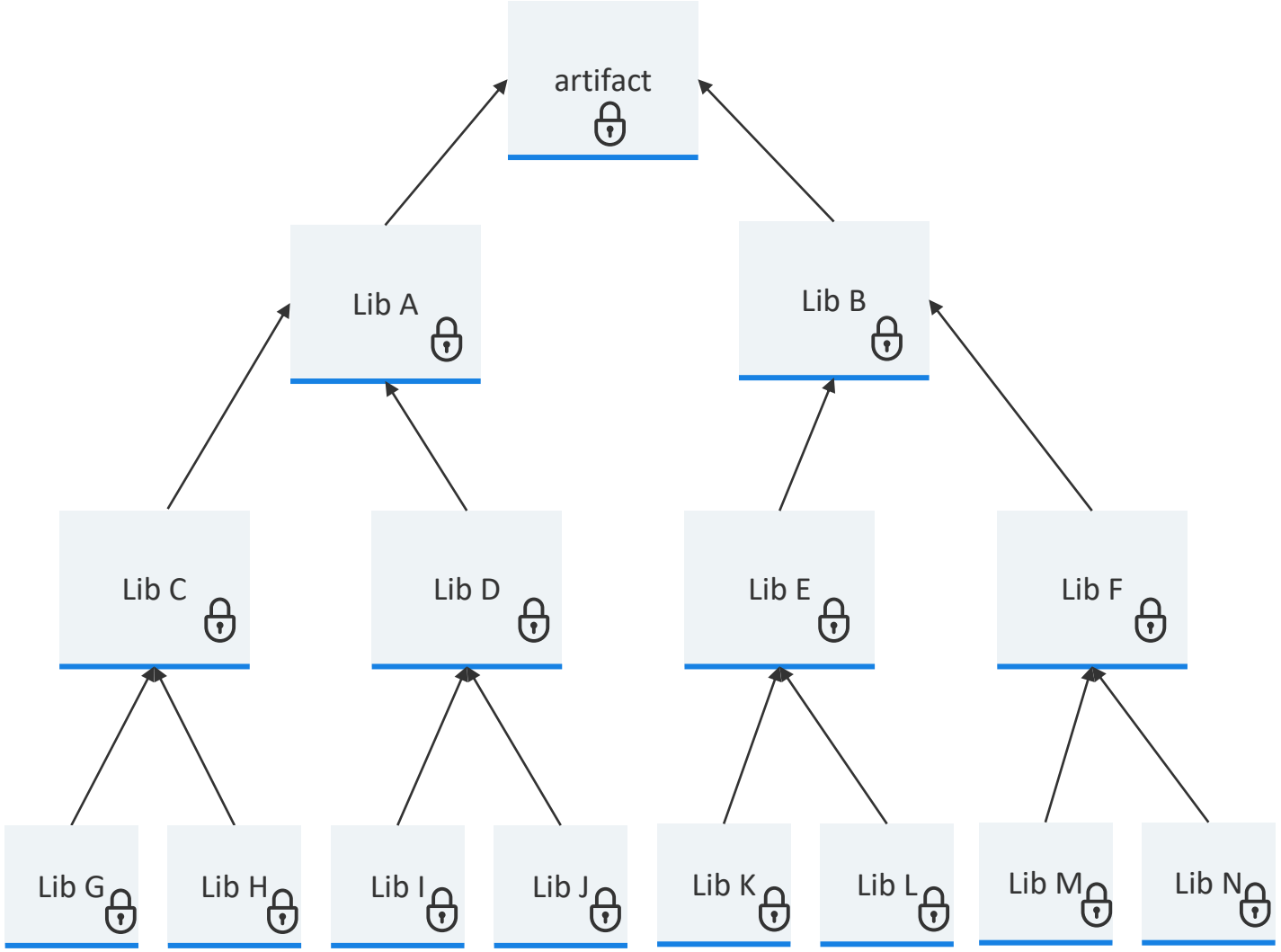


The Importance of the Software Bill of Materials (SBOM)

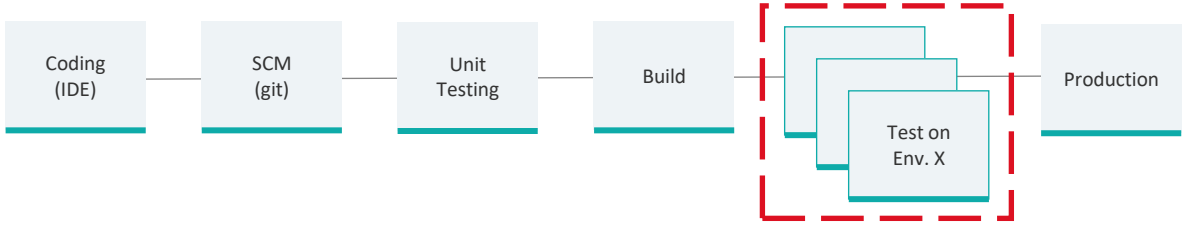
A "**software bill of materials**" (SBOM) has emerged as a key building block in software security and software supply chain risk management.

A SBOM is a nested inventory, a list of ingredients that make up software components.

Source https://www.cisa.gov/sbom

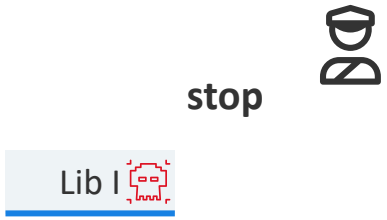


Securing deployed Artifacts



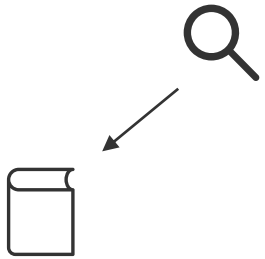
Apply Policies

Secure the deployment through the validation of digital trust (signature). Applying policies that require the progressive series of signed attestations protects environments from accidental, or malicious, artifact deployments. Open Policy Agent can help here.



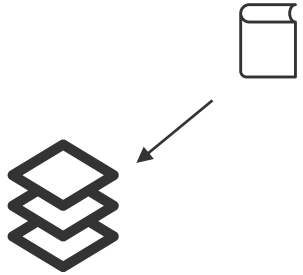
Dynamic Application Security Testing

Dynamic application security testing (DAST) tests are designed to identify functional security vulnerabilities in deployed artifacts. Use DAST to identify possible exploits following known patterns of attack, like SQL injection or cross-site scripting (XSS).



Isolated and replicable testing Environments

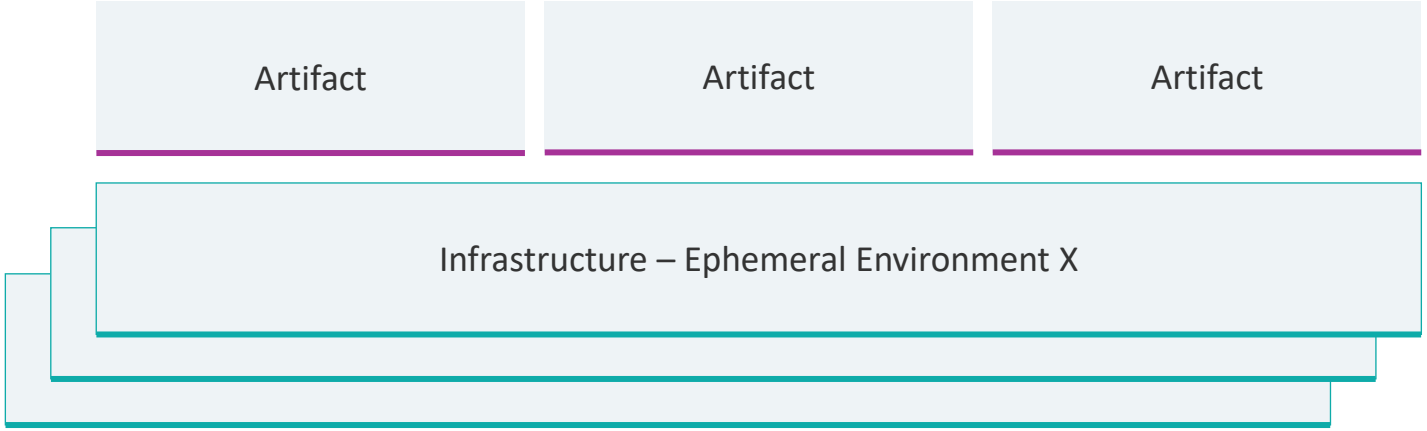
Static environments are challenging. To create environments where you can test new functionality in isolation, use Declarative infrastructure. These environments are ephemeral. They only live as long as the feature test cycle is active.



The importance of declarative Infrastructure (IaC)

A key component of **securing infrastructure** is to use **declarative infrastructure**. Declarative infrastructure is commonly known as **infrastructure as code** (IaC).

It defines infrastructure components as code, manages the components in code repositories, and helps ensure that the infrastructure components undergo the same level of checks and balances as application feature code. Declarative Infrastructure is the basis to manage ephemeral Environments.



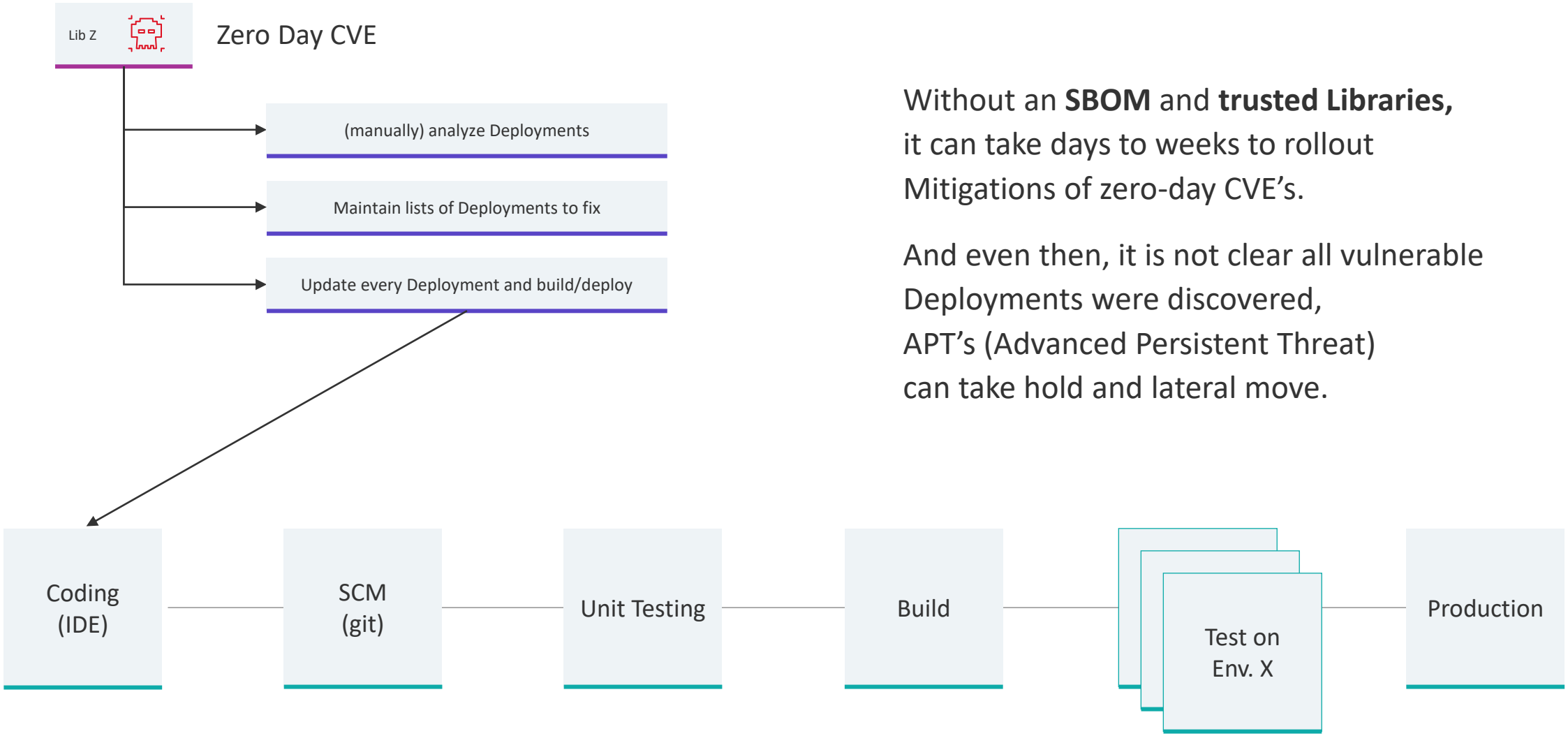
Ephemeral Environment

Environments become replicable. Environments (also prod), can be created or rebuilt automated on demand.

Mitigation of Attacks in the SSDLC

Malicious library Example

Mitigation in a traditional SDLC



Without an **SBOM** and **trusted Libraries,** it can take days to weeks to rollout Mitigations of zero-day CVE's.

And even then, it is not clear all vulnerable Deployments were discovered, APT's (Advanced Persistent Threat) can take hold and lateral move.

Mitigation in a SSDLC with the 3 R's of Enterprise Security

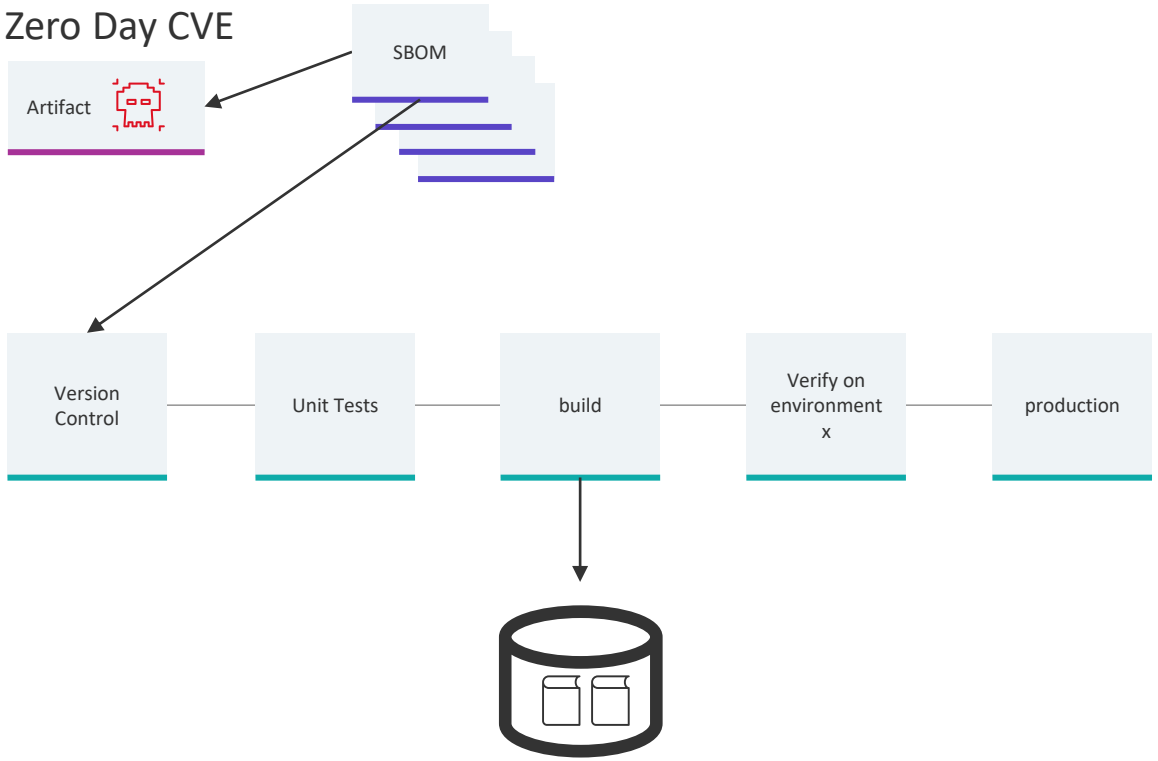
Rotate datacenter credentials every few minutes or hours.

Repair vulnerable operating systems and application stacks consistently within hours of patch availability. Faster is safer.

Repave every server and application in the datacenter every few hours from a known good state.

Source:
Pivotal, ThoughtWorks Techradar

Repair



Requirements for Repair

- CVE tracking
- SBOM and (automated) manifest update
- Signed Artifacts

Mitigation in a SSDLC with the 3 R's of Enterprise Security

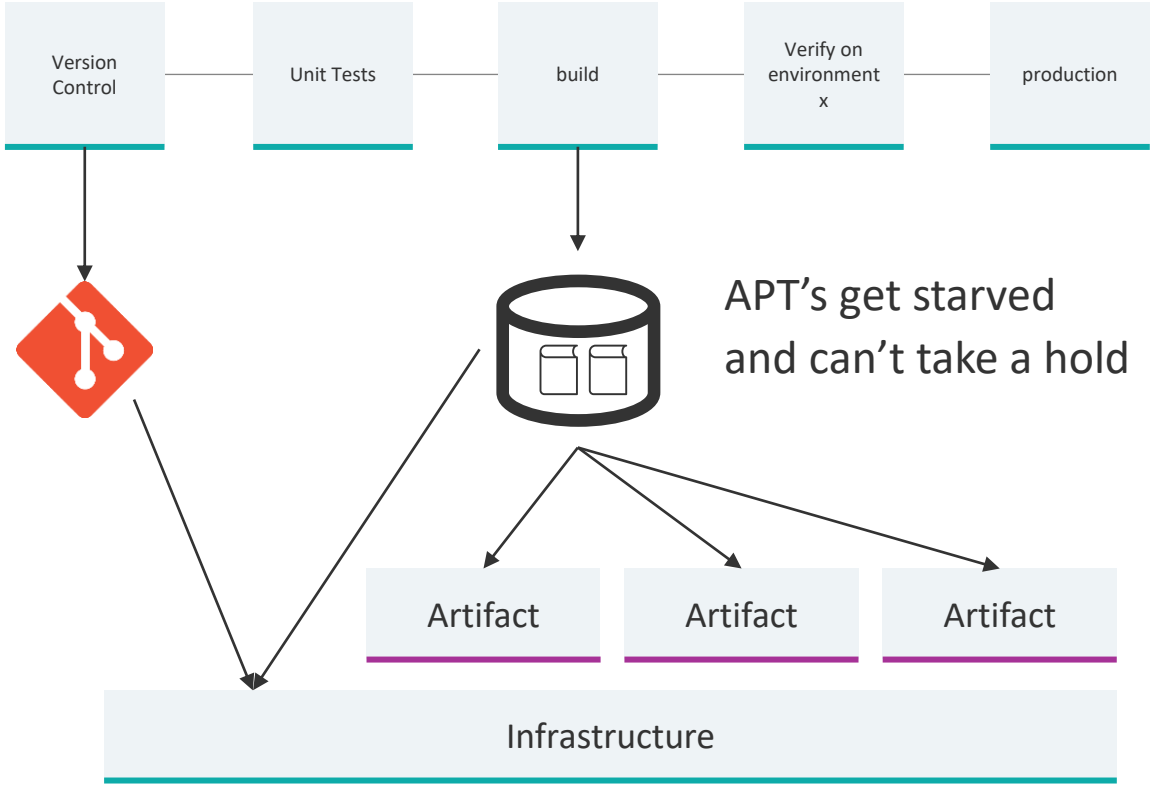
Rotate datacenter credentials every few minutes or hours.

Repair vulnerable operating systems and application stacks consistently within hours of patch availability. Faster is safer.

Repave every server and application in the datacenter every few hours from a known good state.

Source:
Pivotal, ThoughtWorks Techradar

Repave



Requirements for Repave

- Declarative Infrastructure
- Cloud Native / Stateless Architecture
- Known trusted State

ISO 20’000 and SSDLC (Secure Software Development Life Cycle)

the international standard for IT service management

Automated and Auditable Software Supply Chain

The Software Supply Chain must be auditable to ensure ISO 20’000 compliance.
The Implementation of the Software supply chain has to guarantee that the Software supply chain covers integrity and confidentiality.

Every step in the building, integration and testing process has to be secured that integrity and confidentiality is guaranteed from the code commit to the running workload on production.

In a traditional environment this is/was mostly done trough documented human interaction (handover, signoff etc.)

If we apply zero trust to the SDLC and combine it with an audit trail, this requirement is already fulfilled.

Shift left Security – Key Takeaways

Verifiable Identities

Its all about verifiable entities and trust – zero trust applied to the SDLC results in SSDLC

Ephemeral Environments

Ephemeral Environments, immutable artifacts and infrastructure is a must

Standardizes Processes and Tools

Standardize Software Development Processes and Tools enable SSDLC

Change the culture

Change the culture in your workforce towards a security – zero trust culture

Enable the Developers

Make it easy for your Engineers to deliver Software in a secure way

Secure Software Development Lifecycle Journey

where is Swisscom

Threat Modelling

Threat modelling is executed on critical services / applications. Actually this is the “leftest” security measure.

App Sec Peaks

We have a security program, which shifts the security Know How left to the DevOps Teams. Every Team should have a certified Security Champion.

SAST/SBOM with Snyk

Snyk integrates perfect in the Dev Workflow to Scan Code on vulnerabilities and sensitive information.

Artifact repositories

Central repositories are used to control the flow from public repositories and scan inhouse produced artifacts as also external fetched artifacts

isolated ephemeral Environments

Not pervasive, but is used more and more. The dominance of kubernetes helps here.

References

- DevSecOps Booklet Swisscom
- Shifting Left on Security Google
- The Three R’s of Enterprise Security: Rotate, Repave, and Repair

Question and Answer